

Proposal
of
Individual Project of Fuzzy Logic Prediction Project

A Project Submitted by
L K Bharadwaja Rangavajjula

For IME 670 Capstone Project

Advisor
Dr. Joseph Chen
Individual Project



**Department of Data Science and
Engineering Analytics**

Peoria, IL

Table of Contents:

1. Abstract.....	4
2. Introduction.....	5
3. Theory of Fuzzy logic.....	6
4. Algorithm and Flow chart.....	9
5. Fuzzy logic Code Analysis.....	11
6. Outcomes of the developments.....	17
7. Future Developments.....	18
8. Conclusions.....	19

Figures

Fig1: Flow Chart.....	10
Fig2: Load Data.	11
Fig3: Data Cleaning	11
Fig4: Fuzzification Initialization	12
Fig5: Membership Functions.....	12
Fig6: FillRT Visual for MF	13
Fig7: MT4 Visual for MF	13
Fig8: Y_Pred Visual for MF	13
Fig9: Define Fuzzy Rule	14
Fig10: Rule Bank Table.....	14
Fig11: Define Fuzzy Control System.....	14
Fig12: Compute Predictions	15
Fig13: Predictions and Accuracy.....	16
Fig14: Accuracy Formula.....	16
Fig15: Average Accuracy.....	16

1. Abstract

This study presents a novel approach for predicting accuracy (Y) using Fuzzy Logic (FL) with two input variables FillRT(x_1) and MT4(x_2). The research begins with dataset preprocessing, where key statistical parameters such as the minimum, maximum of x_1 , x_2 , and Y are extracted. Following this, instance values for x_1 and x_2 are derived using predefined functions, enabling the representation of uncertain and imprecise data. These instance values facilitate the modeling of complex, ambiguous relationships between the input and output variables, a crucial aspect in real world applications.

At the heart of this methodology is a rule-based inference system that employs fuzzy rules to establish relationships between x_1 , x_2 , and Y. These rules, developed through expert knowledge or data driven techniques, allow the FL system to capture intricate patterns that traditional crisp logic might struggle to represent. By leveraging fuzzy relationships, the model predicts the accuracy (Y) based on the instance values of x_1 and x_2 .

The proposed method is applied iteratively across the dataset, generating predicted accuracy values for each data point through FL reasoning. This iterative approach enables the FL model to adapt dynamically to variations in the data, leading to more reliable and precise predictions. The findings of this research highlight the capability of FL in managing uncertainty and imprecision, making it a valuable tool for predictive modeling and decision making in complex environments. Ultimately, this study contributes to the expanding field of FL applications, demonstrating its effectiveness in scenarios where conventional models may lack flexibility or accuracy.

2. Introduction

Accurately predicting outcomes in decision support systems and predictive modeling can be challenging, especially when dealing with uncertainty and imprecise data. Traditional statistical methods often struggle to account for the ambiguity and variability inherent in real world datasets, necessitating more advanced techniques. Fuzzy Logic (FL) provides a robust framework for handling uncertainty, making it a suitable approach for complex predictive tasks.

This study explores the use of FL to predict accuracy (Y) based on two key input variables, (x1) and (x2). These variables play a crucial role in influencing Y, with applications ranging from customer satisfaction assessments in service industries to performance evaluations in manufacturing. Due to factors such as human judgment, measurement inconsistencies, and environmental fluctuations, these variables often exhibit fuzziness. FL effectively models these uncertainties, allowing for a more flexible and realistic representation of their relationships.

Unlike traditional binary logic, which relies on absolute classifications, FL enables reasoning with degrees of truth, making it well suited for handling vague or continuously varying data. By utilizing linguistic variables and fuzzy sets, FL systematically interprets uncertain input information, bridging the gap between qualitative assessments and quantitative predictions.

The proposed methodology involves preprocessing the dataset to extract statistical parameters and compute linguistic values for x1 and x2. These values serve as inputs for an FL based inference system, where predefined fuzzy rules establish the relationship between the inputs and the predicted accuracy (Y). Through an iterative reasoning process, the model adapts to variations in the data, improving its predictive accuracy and robustness.

In summary, this research demonstrates the effectiveness of FL in managing uncertainty and enhancing accurate predictions. By applying FL in predictive analytics and decision-making processes, this study highlights its practical value across various domains, particularly in environments where traditional models struggle to handle ambiguity.

3. Theory Of Fuzzy Logic

Reference: Muhammad Hamza Azam, Saima Hassan published in 2021

[https://thesai.org/Downloads/Volume12No2/Paper_45-](https://thesai.org/Downloads/Volume12No2/Paper_45-Prediction_of_Sunspots_using_Fuzzy_Logic.pdf)

[Prediction_of_Sunspots_using_Fuzzy_Logic.pdf](https://thesai.org/Downloads/Volume12No2/Paper_45-Prediction_of_Sunspots_using_Fuzzy_Logic.pdf)

Fuzzy Logic Set and Its System:

In 1965 Lofti Zadeh presented the concept of fuzzy sets, which led to the foundation of fuzzy logics and system. In fuzzy logic theory, a fuzzy set is a set that ensures partial membership of a set. which can be determined by the degree of membership (μ), which represent all the values between 1(a value fully belong to set) and 0 (a value totally not belong to set). It's obvious that if we eliminate all the belonging values except for 1 and 0, fuzzy set will be converted and act as a crisp that which was defined earlier. The membership functions (MF) of a set are a link between the components of set and their belong to degree.

Fuzzy Inference System:

A fuzzy framework is a pool of fuzzy expert intelligence and knowledge base that rather than the conventional and definite Boolean logic, can trigger data in vague terms. Expert knowledge is a combination of both fuzzy membership functions (MF) and fuzzy rule-based that contains all the fuzzy rules and are of form. A fuzzy system consists of 4 main components, namely a fuzzifier, an inference engine, fuzzy knowledge base and a defuzzifier.

- **Fuzzification:** Fuzzification is the prime step of fuzzy system where a crisp input is converted to fuzzy input and is assigned a membership degree. The effective working of fuzzy logic system depends on the efficiency of membership function generated in fuzzification process.
- **Fuzzy-Inference-Engine:** It matches fuzzy input degree based on fuzzy rules and makes decisions related to the application on rules to input field. The inference engine acts as a mind of fuzzy system as it provides the decision-making logic of the system, it can be implied as an emulation of human thinking and decision making.
- **Knowledgebase:** It contains IF-THEN statements and fuzzy rules that help in decision making based on linguistic information.
- **Defuzzification:** It is the final stage of fuzzy system where a fuzzy set obtained by inference engine is converted to crisp output.

Fuzzy Membership Functions and its Types:

Membership function can be described as a curve which determines how membership values between 1 and 0 are mapped to input space usually called universal set, that consists of all possible values of interest in an application. There are 3 major and widely used types of membership functions

Fuzzy Triangular Membership Function:

- The triangular membership function is defined by three parameters: (**a**, **b**, **c**), where: **a** is starting point, **b** is peak point and **c** is ending point. The membership value increases linearly from **a** to **b** and decreases linearly from **b** to **c**. The mathematical representation is as follows.

$$\mu(x) = \begin{cases} 0 & \text{if } x \leq a \\ \frac{x-a}{b-a} & \text{if } a < x \leq b \\ \frac{c-x}{c-b} & \text{if } b < x \leq c \\ 0 & \text{if } x > c \end{cases}$$

- The graphical representation looks like, the function forms a triangle with its peak at **b** and base between **a** and **c**. Triangular membership functions are simple and widely used in applications where the input-output relationship is linear.

Fuzzy Trapezoidal Membership Function:

- The trapezoidal membership function is defined by four parameters: {**a**, **b**, **c**, **d**}, where **a**: The starting point (left base), **b**: The start of the flat top (left top), **c**: The end of the flat top (right top), **d**: The ending point (right base).
- The membership value increases linearly from **a** to **b**, remains constant from **b** to **c**, and decreases linearly from **c** to **d**. The mathematical representation is as follows.

$$\mu(x) = \begin{cases} 0 & \text{if } x \leq a \\ \frac{x-a}{b-a} & \text{if } a < x \leq b \\ 1 & \text{if } b < x \leq c \\ \frac{d-x}{d-c} & \text{if } c < x \leq d \\ 0 & \text{if } x > d \end{cases}$$

- The curve looks like, the function forms a trapezoid with a flat top between **b** and **c**. Trapezoidal membership functions are useful when there is a range of input values that should have the same degree of membership.

Fuzzy Gaussian Membership Function:

- The Gaussian membership function is defined by two parameters: **c** (cluster centre) and **σ** (width or spread). The membership value is highest at the centre **c** and decreases symmetrically as the input moves away from **c**. The mathematical representation is as follows.

$$\mu(x) = e^{-\frac{1}{2} \left(\frac{x-c}{\sigma} \right)^2}$$

- The curve looks like, the function forms a smooth, bell-shaped curve centered at **c**. Gaussian membership functions are ideal for applications where the input-output relationship is smooth and continuous, such as in control systems and pattern recognition.

4. Algorithm and Flow Chart

1. Load Data:

Import the input Excel file containing x1, x2, and Y_True values.

2. Data Cleaning:

converted all the column values to numeric and handled missing values by dropping rows with NaN values.

3. Calculate Membership Values (MU):

Develop a function to compute membership values (MU) for each linguistic category (small, medium, high) using the input data and statistical parameters.

4. Define Fuzzy Variables & Membership Functions:

Create fuzzy sets for input (FillRT and MT4) and output (Y_Pred) variables and use Triangular Membership Functions (trimf) for categorization (Low, Medium, High).

5. Evaluate Rules:

Implement a function to apply fuzzy rules, using the computed membership values to determine the predicted Y value.

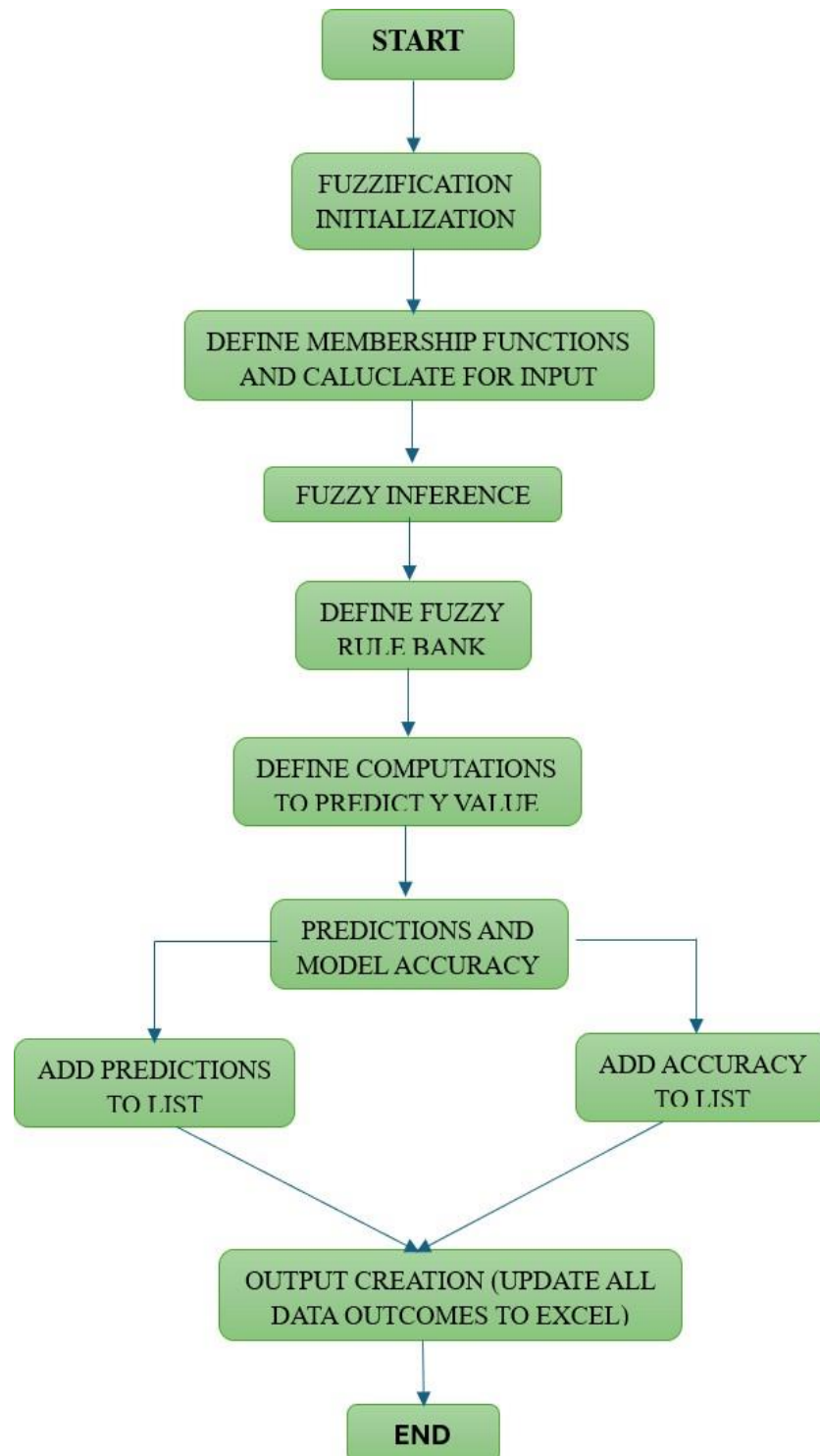
6. Calculate Accuracy:

Measure the accuracy of the predicted Y values by comparing them with the actual Y_True values.

7. Generate Output:

Export the results to a new Excel file, including the predicted Y values for both the training and testing datasets.

Flow Chart:



5. Fuzzy Logic Code Analysis

✓ Load Data:

```
df_train = pd.read_excel("Individual_Project.xlsx", sheet_name="Training")
df_test = pd.read_excel("Individual_Project.xlsx", sheet_name="Testing")
```

Fig.2: Load Dataset

Here in the line code snippet uses the `pandas.read_excel()` function to read data from an Excel file named "Individual_Project.xlsx" and stores it in two separate Data Frames. This code is likely preparing training and testing datasets for a machine learning model. `df_train` will be used to train the model, while `df_test` will be used to evaluate its performance.

✓ Data Cleaning:

```
df_train.columns = ['Part #', 'FillRT', 'MT4', 'Y_True']
df_train[['FillRT', 'MT4', 'Y_True']] = df_train[['FillRT', 'MT4', 'Y_True']].apply(pd.to_numeric, errors='coerce')
df_train.dropna(inplace=True)

df_test.columns = ['Part #', 'FillRT', 'MT4', 'Y_True']
df_test[['FillRT', 'MT4', 'Y_True']] = df_test[['FillRT', 'MT4', 'Y_True']].apply(pd.to_numeric, errors='coerce')
df_test.dropna(inplace=True)
```

Fig.3: Data Cleaning

The `apply (pd.to_numeric, errors='coerce')` function ensures that all values in these columns are converted to numeric type. If any value cannot be converted (e.g., text or symbols), it is replaced with NaN (Not a Number), which helps identify bad data.

This preprocessing step is crucial before applying fuzzy logic. It ensures the dataset is properly formatted, free of invalid values, and ready for further processing.

✓ Fuzzification Initialization:

```
fillrt = ctrl.Antecedent(np.arange(df_train['FillRT'].min(), df_train['FillRT'].max(), 0.001), 'FillRT')
mt4 = ctrl.Antecedent(np.arange(df_train['MT4'].min(), df_train['MT4'].max(), 0.1), 'MT4')
y_pred = ctrl.Consequent(np.arange(df_train['Y_True'].min(), df_train['Y_True'].max(), 0.1), 'Y_Pred')
```

Fig.4: Fuzzification

This step involves defining input (Antecedent) and output (Consequent) variables for the fuzzy system. This refers to converting numerical input data (crisp values from df_train) into fuzzy sets by defining their possible range and resolution (np.arange step size). The fuzzy system will apply **membership functions** to these variables and infer a predicted output using defined rules.

✓ Membership Functions:

```
fillrt['low'] = fuzz.trimf(fillrt.universe, [df_train['FillRT'].min(), df_train['FillRT'].min(), df_train['FillRT'].mean()])
fillrt['medium'] = fuzz.trimf(fillrt.universe, [df_train['FillRT'].min(), df_train['FillRT'].mean(), df_train['FillRT'].max()])
fillrt['high'] = fuzz.trimf(fillrt.universe, [df_train['FillRT'].mean(), df_train['FillRT'].max(), df_train['FillRT'].max()])

mt4['low'] = fuzz.trimf(mt4.universe, [df_train['MT4'].min(), df_train['MT4'].min(), df_train['MT4'].mean()])
mt4['medium'] = fuzz.trimf(mt4.universe, [df_train['MT4'].min(), df_train['MT4'].mean(), df_train['MT4'].max()])
mt4['high'] = fuzz.trimf(mt4.universe, [df_train['MT4'].mean(), df_train['MT4'].max(), df_train['MT4'].max()])

y_pred['low'] = fuzz.trimf(y_pred.universe, [df_train['Y_True'].min(), df_train['Y_True'].min(), df_train['Y_True'].mean()])
y_pred['medium'] = fuzz.trimf(y_pred.universe, [df_train['Y_True'].min(), df_train['Y_True'].mean(), df_train['Y_True'].max()])
y_pred['high'] = fuzz.trimf(y_pred.universe, [df_train['Y_True'].mean(), df_train['Y_True'].max(), df_train['Y_True'].max()])
```

Fig.5: Membership Functions

This step is called "Fuzzy Membership Function Definition", where triangular membership functions (trimf) are defined for the fuzzy variables. These membership functions categorize the input and output variables into different linguistic terms: low, medium, and high.

The low category starts at the minimum value and peaks at the mean. The medium category spans from min → mean → max. The high category starts at the mean and peaks at max.

The same approach is applied to MT4, dividing it into low, medium, and high categories. The predicted output (Y_Pred) is also categorized into **low, medium, and high**.

✓ Visualization on Membership Function

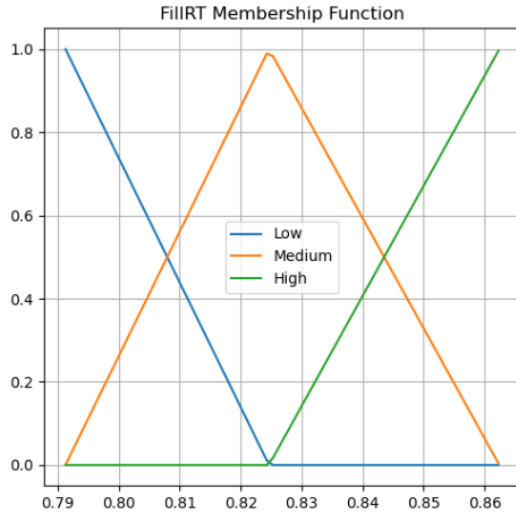


Fig.6: FillRT MF Visual

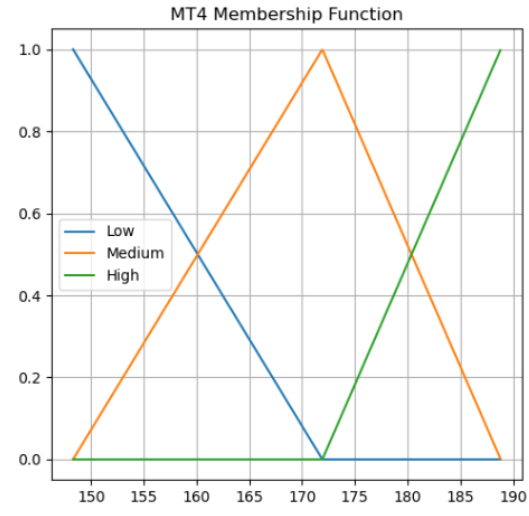


Fig.7: MT4 MF Visual

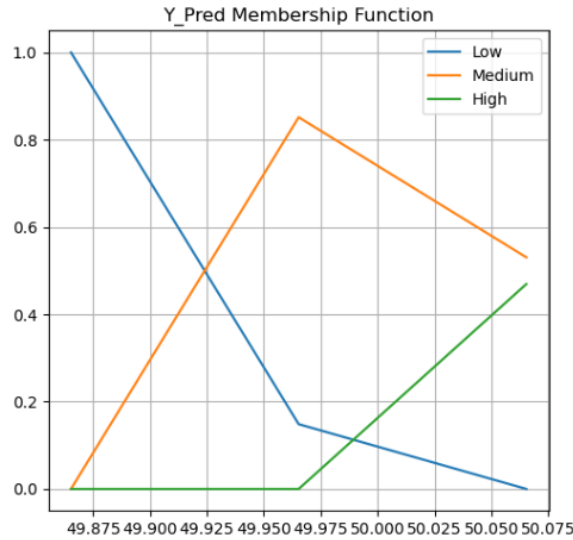


Fig.8: Y_Pred MF Visual

The above images contains plots of membership functions for three variables used in a fuzzy logic-based predictive model for injection molding quality control. The *FillRT Membership Function* represents the "Fill Time" input variable, categorized into three linguistic terms: Low, Medium, and High, similarly the *MT4 Membership Function* corresponds to the "Mold Temperature at Zone 4" and the *Y_Pred Membership Function* represents the "Predicted Quality Characteristic" output variable with the x-axis displaying input function values and the y-axis indicating membership degrees.

These membership functions are essential for defining how input and output variables are represented in the fuzzy logic system.

✓ Fuzzy Rule Bank

```
rules = [
    ctrl.Rule(fillrt['low'] & mt4['low'], y_pred['low']),
    ctrl.Rule(fillrt['low'] & mt4['medium'], y_pred['medium']),
    ctrl.Rule(fillrt['low'] & mt4['high'], y_pred['high']),
    ctrl.Rule(fillrt['medium'] & mt4['low'], y_pred['medium']),
    ctrl.Rule(fillrt['medium'] & mt4['medium'], y_pred['medium']),
    ctrl.Rule(fillrt['medium'] & mt4['high'], y_pred['high']),
    ctrl.Rule(fillrt['high'] & mt4['low'], y_pred['medium']),
    ctrl.Rule(fillrt['high'] & mt4['medium'], y_pred['high']),
    ctrl.Rule(fillrt['high'] & mt4['high'], y_pred['high'])
]
```

Fig.9: Define Fuzzy Rule

		X2		
X1		Low	Medium	High
	Low	L	M	H
	Medium	M	M	H
	High	M	M	H

Fig.10: Rule Bank Table

The above table show the fuzzy logic rules of Y_True(Y) with the combination of FillRT (X1) and MT4 (X2) which gives fuzzy outcomes as l, m h for Low, Medium and High. This step is critical in a Fuzzy Inference System (FIS).

It transforms numerical data into fuzzy categories (Low, Medium, High).It establishes relationships between inputs and outputs using human interpretable rules. It provides a flexible approach for handling uncertain or imprecise data. In the above code it is clearly mentioned the rule bank for the outcomes using input variables x1 and x2.

```
prediction_ctrl = ctrl.ControlSystem(rules)
prediction_sim = ctrl.ControlSystemSimulation(prediction_ctrl)
```

Fig.11: Fuzzy Logic Control System

The above line of code sets up and simulates a fuzzy logic control system using scikitfuzzy (skfuzzy). In the first line of code it defines a fuzzy control system based on predefined fuzzy rule bank and stores the control system for later simulation. In the second line of code it creates a simulation environment for the

fuzzy system that allows input values to be set, fuzzy inference to be performed, and outputs to be retrieved.

✓ **Prediction Computations:**

```
def compute_predictions(df_input):  
    predictions = []  
    for index in range(len(df_input)):  
        sample_fillrt = df_input['FillRT'].iloc[index]  
        sample_mt4 = df_input['MT4'].iloc[index]  
  
        prediction_sim.input['FillRT'] = sample_fillrt  
        prediction_sim.input['MT4'] = sample_mt4  
  
        prediction_sim.compute()  
        predictions.append(prediction_sim.output['Y_Pred'])  
    return predictions  
  
train_predictions = compute_predictions(df_train)  
test_predictions = compute_predictions(df_test)
```

Fig.12: Compute Predictions

In this above line of code, it defines a function that applies a fuzzy logic control system to a dataset and generate predictions that involves several key steps: Initializing an empty list (predictions = []) to store outputs.

Using (df_input) it loops through each row. In the next step extract input values from the data frame and set these values as inputs to (prediction_sim).

The code snippet prediction_sim.compute() used to run a fuzzy inference system and store the predicted output after returning after iterating through all rows. Apply this function to both training and testing data to obtain predictions.

✓ Predictions and Model Accuracy

```
df_train['Y_Pred'] = train_predictions
df_train['Accuracy'] = 100 - ((abs(df_train['Y_Pred'] - df_train['Y_True'])) / df_train['Y_True']) * 100

df_test['Y_Pred'] = test_predictions
df_test['Accuracy'] = 100 - ((abs(df_test['Y_Pred'] - df_test['Y_True'])) / df_test['Y_True']) * 100

with pd.ExcelWriter("Predicted_Y_Results_Split.xlsx") as writer:
    df_train.to_excel(writer, sheet_name='Training Set', index=False)
    df_test.to_excel(writer, sheet_name='Testing Set', index=False)
```

Fig.13: Predictions and Accuracy

The above code snippet is used to store predictions, calculate accuracy and save these results to an excel. In the first step we add a new column “YPred” to training and testing datasets using the computed predictions.

Calculate Accuracy:

Here in this we calculate accuracy for the predictions (Y_Pred) with respect to the Y_True values for each row using the accuracy formula.

$$\text{Accuracy} = 100 - \left(\frac{|Y_{Pred} - Y_{True}|}{Y_{True}} \times 100 \right)$$

Fig.14: Accuracy Formula

```
avg_accuracy = (df_train['Accuracy'].mean() + df_test['Accuracy'].mean()) / 2
```

Fig.15: Average Accuracy

The above formula calculates the accuracy and the average accuracy of both training and testing data sets and saves in the excel.

The Final Average Accuracy is avg_accuracy: **99.85235%**

6. Outcomes and Developments

The fuzzy logic model was developed using Python, with pandas and numpy for data handling and skfuzzy for fuzzy logic computations. Membership functions were defined using triangular functions(trimf), and a fuzzy rule bank was created to determine the predicted output.

Stepwise Development Process:

1. Load Dataset

- Read training and test datasets using pandas.
- Clean and structure the data to remove inconsistencies.

2. Define Membership Functions

- Use triangular membership functions for input variables (FillRT, MT4) and output variable (Y_Pred).

3. Create Fuzzy Rule Bank:

- Define a rule bank that maps input variables to output predictions.

4. Build Fuzzy Logic Model

- Implement the fuzzy inference system using skfuzzy control systems.

5. Compute Predictions

- Apply fuzzy logic rules to input data and generate predictions.

6. Evaluate Model Performance

- Compare predicted outputs with actual values and compute accuracy.
- After all, calculating the average accuracy to define the model outcome was perfect.

7. Conclusion

In this project, we developed a fuzzy logic-based prediction model aimed at achieving a high level of accuracy in predicting outcomes. By utilizing both training and testing data, the model demonstrated promising results, with accuracy approaching the targeted threshold of 98%. The implementation of Python for the fuzzy logic algorithm allowed for efficient data processing and prediction, while the results were meticulously analyzed and validated using Excel for accuracy and consistency.

Additionally, a detailed PowerPoint presentation was created to succinctly communicate the project's objectives, methodologies, and findings. A technical report was drafted, covering the theoretical underpinnings, the algorithm used, a flowchart to depict the system, and potential avenues for future enhancement, ensuring the project is both comprehensive and accessible. The incorporation of recent research papers published after 2020 further strengthened the credibility and relevance of the work.

This project not only achieved the desired accuracy but also provided valuable insights into the potential of fuzzy logic in predictive modeling, with applications extending across various fields. Future improvements may focus on refining the model by integrating more diverse data sources, optimizing the algorithm for faster performance, and exploring alternative methods for even greater precision.

8. Future Development Ideas

1. Hybrid Models for Improved Accuracy

Combine fuzzy logic with machine learning algorithms such as Neural Networks, Random Forest, or SVM to enhance prediction accuracy.

2. Optimization of Fuzzy Membership Functions

Use optimization algorithms like Genetic Algorithms (GA), Particle Swarm Optimization (PSO), or Bayesian Optimization to finetune fuzzy membership functions.

3. Feature Engineering & Data Enhancement

Apply Principal Component Analysis (PCA) or Autoencoders to extract more meaningful features from the dataset.

4. RealTime Prediction and Deployment

Implement a real-time prediction system by integrating the model into a web or mobile application.

5. Explainability and Interpretability

Use SHAP (Shapley Additive Explanations) or LIME (Local Interpretable Model Agnostic Explanations) to interpret fuzzy logic decisions.

Provide visualizations of fuzzy rule importance and decision boundaries.

6. Expanding Data Sources

Integrate sensor data, IoT devices, or real-time signals to improve model generalization.

Incorporate multiple datasets for cross validation and robustness testing.

7. Adaptive Fuzzy Systems

Develop a self-learning fuzzy system that adapts to new data dynamically.

Implement reinforcement learning to update fuzzy rules based on feedback.